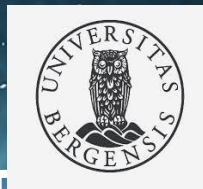


Fine-Tuning LLMs with Multi-GPU Training in an HPC System



Norwegian research infrastructure services

Hicham Agueny, PhD
Scientific Computing Group
Info. Tech. Department, UiB/NRIS



PART III

Hands-on:

Fine-Tuning on a single GPU + Inference

Dataset: Alpaca dataset

Question Answering tasks

Time:

Hands-on: Fine-Tuning on a single GPU + Inference

Dataset: Alpaca dataset: Question Answering (QA) tasks

```
dataset:  
  _component_: torchtune.datasets.alpaca_cleaned_dataset  
  packed: False  
seed: null  
shuffle: True  
batch_size: 4
```

- **Hands-On Workflow**

Instructions for Fine-tuning on a single GPU: [fine-tuning-singlegpu/README.md](#)

Instructions for inference: [inference/README.md](#)

1. Explore SLURM job scripts.
2. Review configuration files for LoRA FT.
3. Submit jobs and monitor GPU usage.
4. Inspect logs, metrics, and visualize results.
5. Perform inference on QA task.



Hands-on:

Fine-Tuning on multiple GPUs + Inference

Dataset: Alpaca dataset

Question Answering tasks

Hands-on: Fine-Tuning on multi GPUs + Inference

Dataset: Alpaca dataset: Question Answering (QA) tasks

```
dataset:  
  _component_: torchtune.datasets.alpaca_cleaned_dataset  
  packed: False  
seed: null  
shuffle: True  
batch_size: 4
```

- **Hands-On Workflow**

Instructions for Fine-tuning on a single GPU: [fine-tuning-multigpu/README.md](#)

Instructions for inference: [inference/README.md](#)

1. Explore SLURM job scripts.
2. Review configuration files for LoRA FT.
3. Submit jobs and monitor GPU usage.
4. Inspect logs, metrics, and visualize results.
5. Compare metrics: single GPU vs multiple GPUs



Hands-on:

Profiling: Fine-Tuning on a single GPU

Profiling: Fine-Tuning on a single GPU

```
# Profiler (enabled)
```

```
profiler:
```

```
  _component_: torchtune.training.setup_torch_profiler
```

```
  enabled: True
```

```
# Output directory for trace artifacts
```

```
output_dir: /cluster/projects/nn9970k/$USER/llm-workshop/data/profiling_outputs
```

```
# Activities to trace
```

```
cpu: True
```

```
cuda: True
```

```
# Trace options
```

```
profile_memory: True
```

```
with_stack: False
```

```
record_shapes: True
```

```
with_flops: True
```

```
# Scheduling options
```

```
wait_steps: 5
```

```
warmup_steps: 3
```

```
active_steps: 2
```

```
num_cycles: 1
```

Extracted from:

1B_lora_single_device_QA_profiling.yaml

Profiling: Fine-Tuning on a single GPU

- Hands-On Workflow

Instructions for profiling: [profiling/README.md](#)

- 1.Explore SLURM job scripts.
- 2.Review configuration files for profiling.
- 3.Submit a job.
- 4.Visualize results.

Memory Profiling: Fine-Tuning on a single GPU



Hands-on:

Fine-Tuning on a Single GPU + Inference

Dataset: Xsum dataset

Summarization tasks

Hands-on: Fine-Tuning on a single GPU/multiGPUs + Inference

Dataset: Xsum dataset:
Summarization tasks

- **Hands-On Workflow**

Instructions for Fine-tuning

on a single GPU & multi GPUs:

[exercise/README.md](#)

1. Explore SLURM job scripts.
2. Review configuration files for LoRA FT.
3. Submit jobs and monitor GPU usage.
4. Inspect logs, metrics, and visualize results.
5. Perform inference on Summarization task.

```
dataset:
  _component_: torchtune.datasets.instruct_dataset

  # Dataset loading parameters (passed to HF load_dataset)
  source: json
  data_files: /cluster/work/projects/nn9970k/$USER/llm-workshop/data/XSum/xsum.json
  split: train[:2000]

  # TorchTune-specific parameters
  column_map:
    input: document
    output: summary
  train_on_input: false

template:
  _component_: torchtune.data.SummarizeTemplate
```



Wrap up & Discussion

Throughput per-GPU:

Model: Llama 3.2-1B

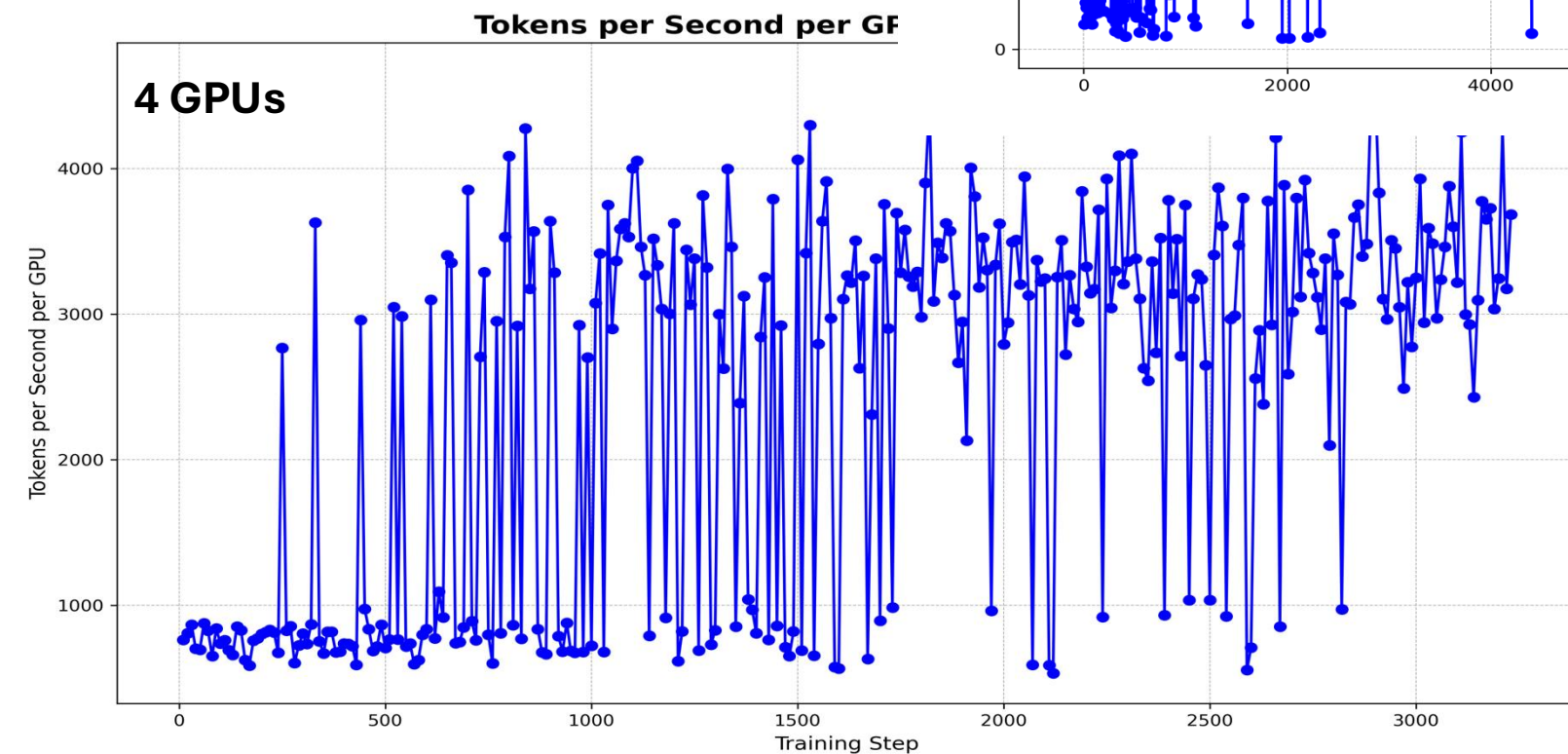
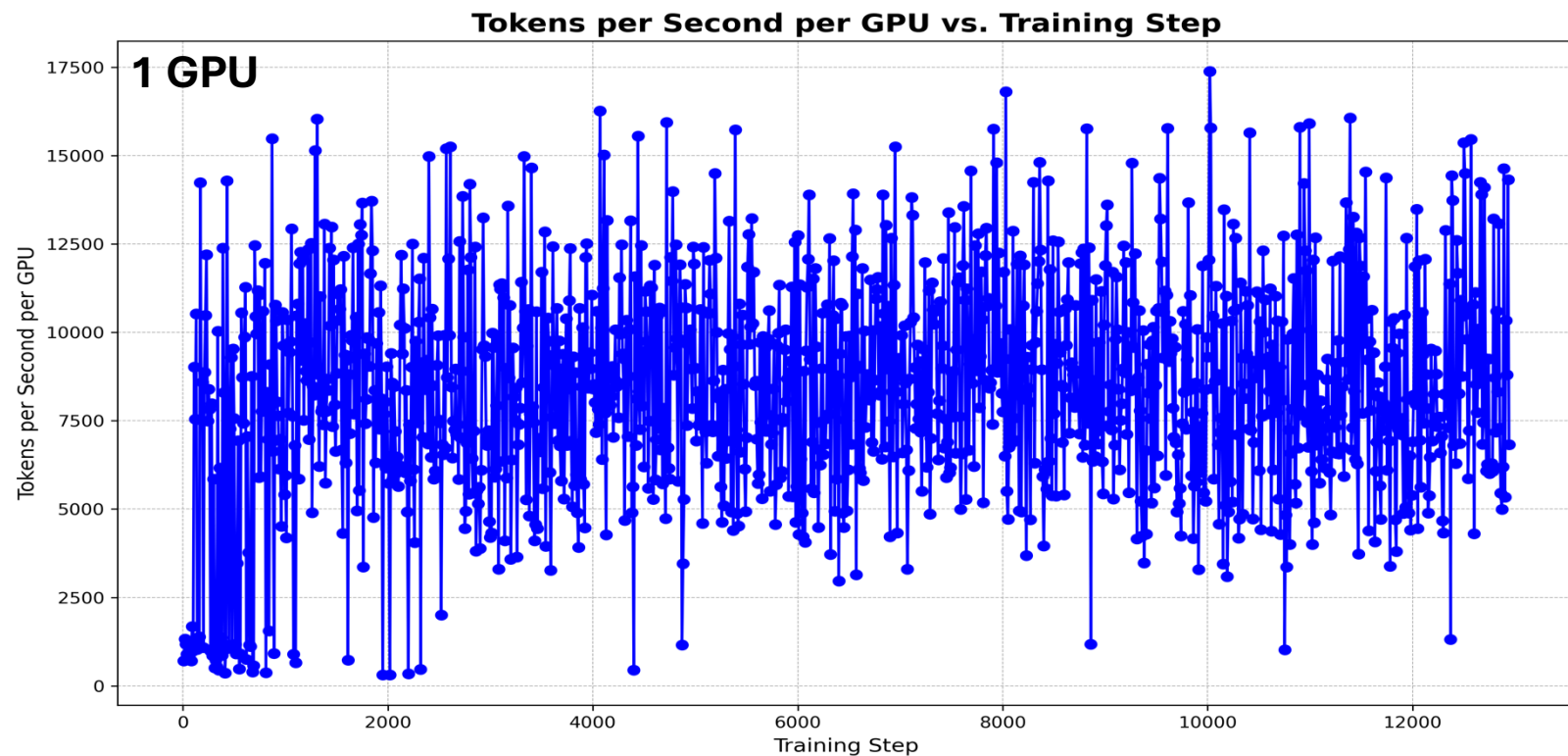
1 GPU: up to **13000** Tokens per sec per GPU

4 GPUs: up to **4000** Token per sec per GPU

Total system throughput : $4 \times 4000 = 16000$

Runtime (1 GPU): 0.12 s/training step

Runtime (4 GPUs): 0.10 s/training step



Parallel efficiency of $\approx 30\%$

1B model, doesn't benefit from multi-GPU training

A single GPU can easily fit the model in memory and process batches efficiently.

Additional overheads in multiGPU setup

Throughput per-GPU:

Model: Llama 3.2-11B

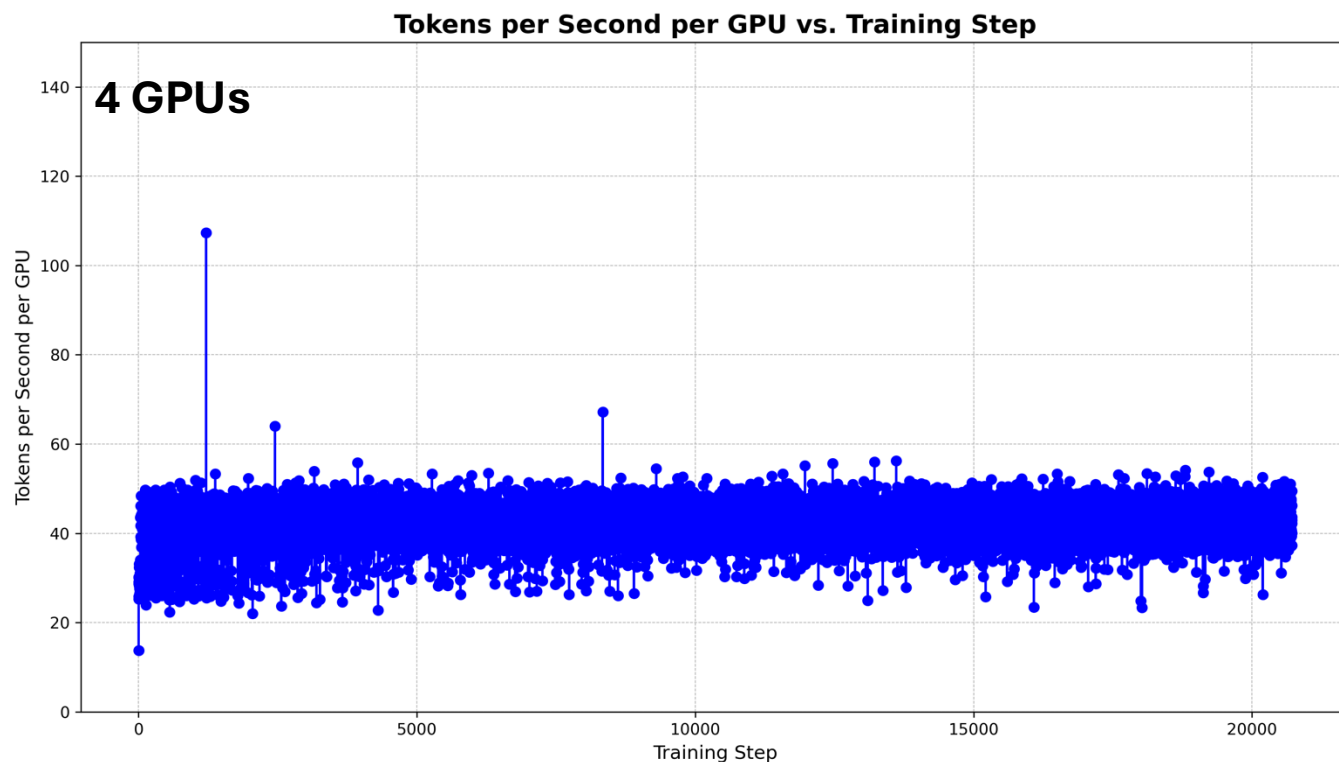
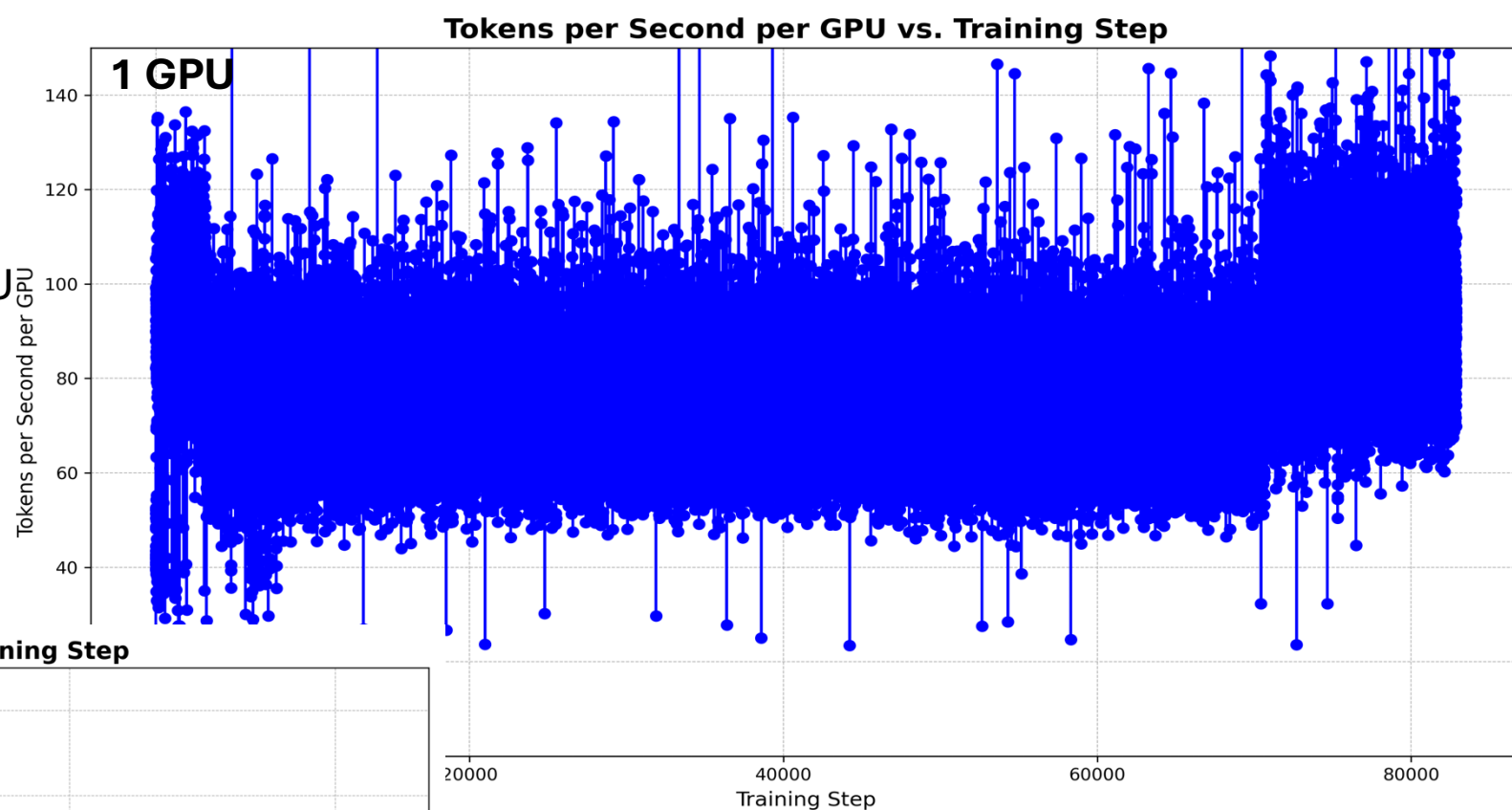
1 GPU: up to **100-120** Tokens per sec per GPU

4 GPUs: up to **50** Token per sec per GPU

Total system throughput : $4 \times 50 = 200$

Runtime (1GPU): 1512m33s(**25.20 h**)

Runtime (4 GPUs): 542m17s (**9.04 h**)



Scaling efficiency:

Parallel efficiency of $\approx 70\%$

Using **4 GPUs** reduced the training time by a **factor of 2.8**

For much larger models, where the model itself cannot fit on a single GPU.....

Throughput per-GPU:

Model: **Llama 3.2-11B**

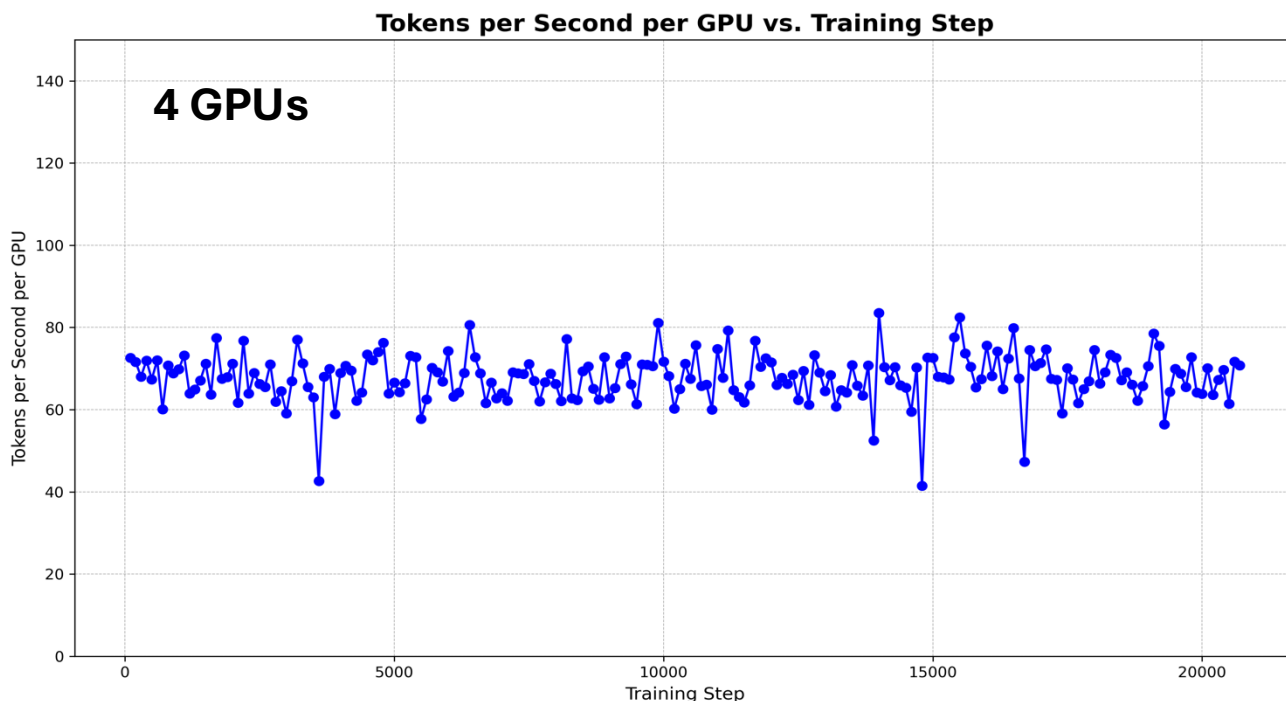
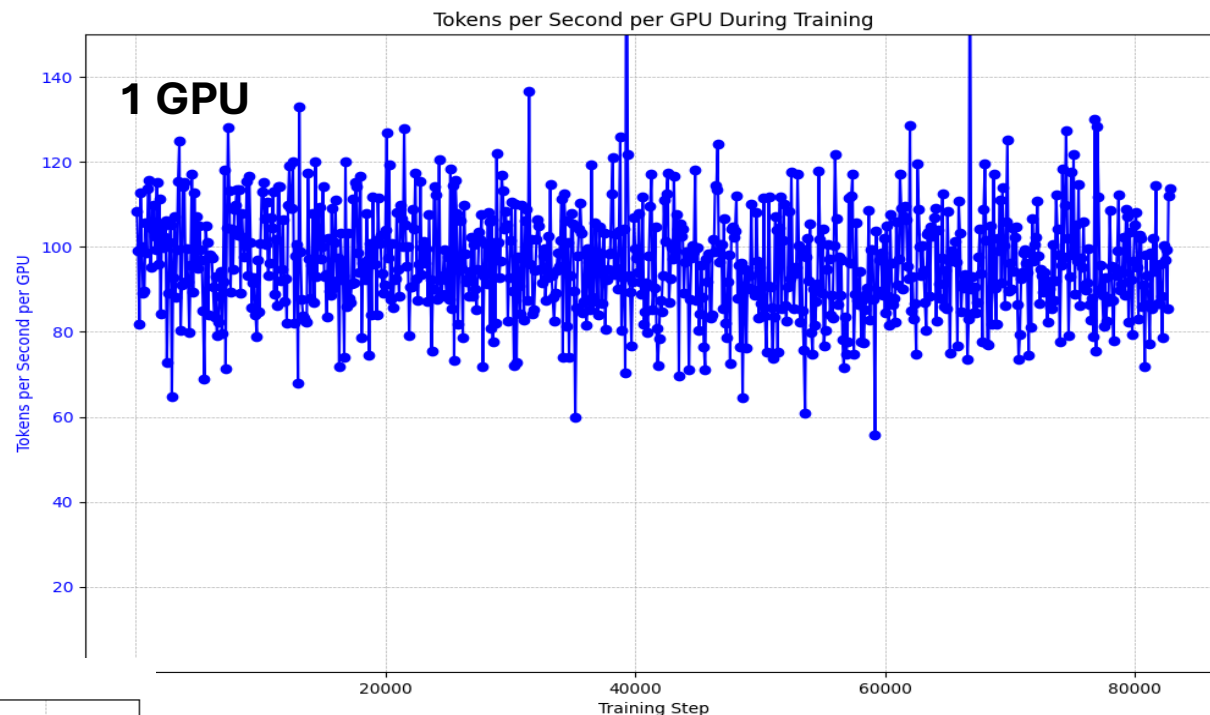
1 GPU: up to **110** Tokens per sec per GPU

4 GPUs: up to **70** Token per sec per GPU

Total system throughput : $4 \times 70 = 280$

Runtime (1GPU): 1.1 second/training step

Runtime (4 GPUs): 0.4 s/training step



Scaling efficiency:

Parallel efficiency of $\approx 70\%$ on 4 GPUs

Using **4 GPUs** reduced the training time by a **factor of 2.8**

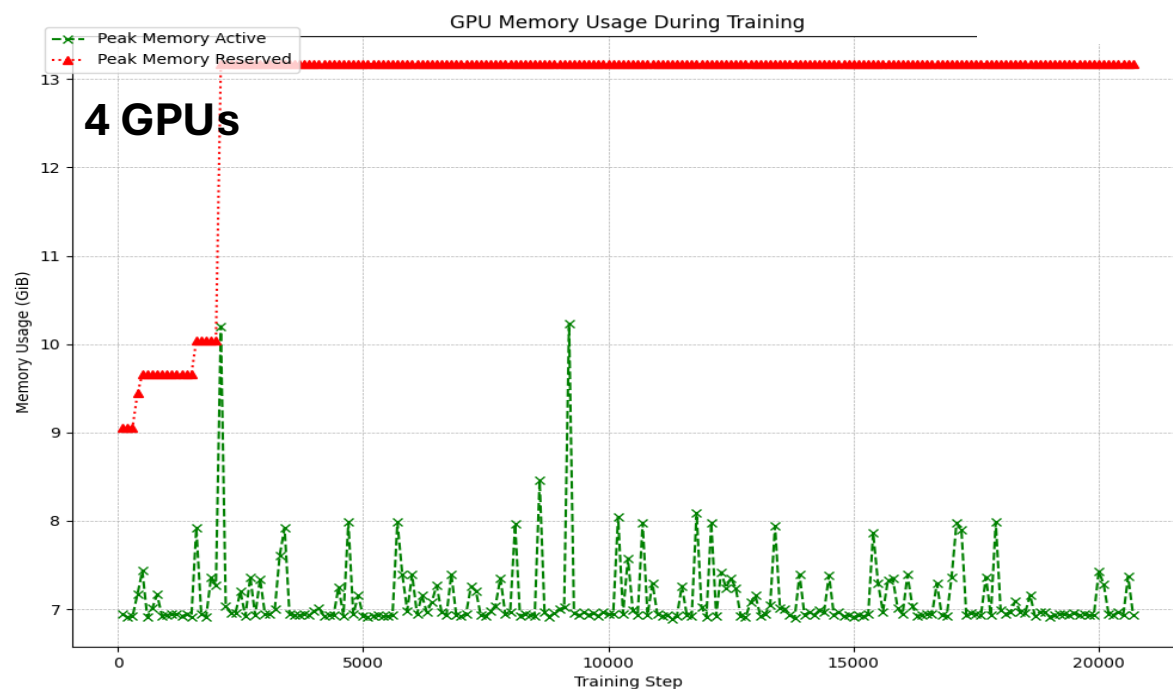
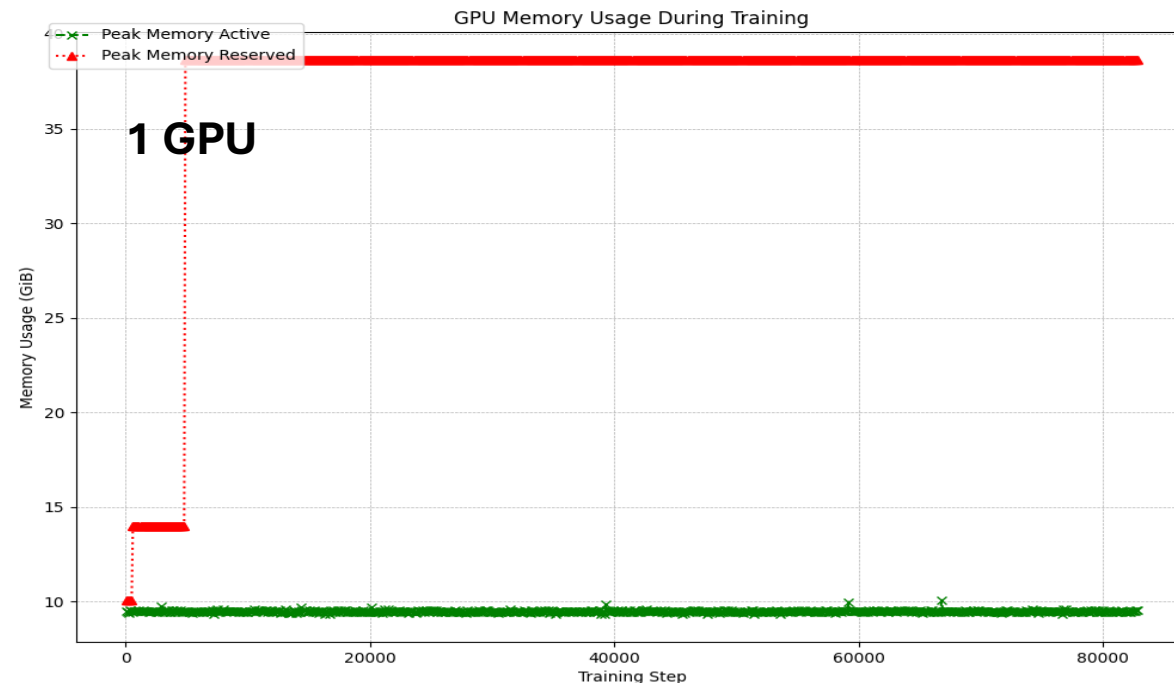
For much larger models, where the model itself cannot fit on a single GPU.....

Memory per-GPU:

Model: Llama 3.2-11B

1 GPU: Peak mem active is **9-10 GiB**
peak mem reserved is up to **38.6 GiB**

4 GPUs: Peak mem active is **7-8 GiB**
Peak mem reserved is up to **13.2 GiB**



Moving from **1 GPU** to **4 GPUs** **reduces** the **peak reserved memory** by a **factor of 3**.
(but slightly lowering the peak active memory)

Efficient distribution of memory across the GPUs.

What's Next ?

Planned Workshop on AI & related topic

To include e.g.:

- DoRA and other optimization techniques**
- Pre-processing of data**
- Model Evaluation**
- Distributed training across multiple nodes**

References

- **GitHub repo LLM-workshop:** <https://github.com/HichamAgueny/llm-workshop>
- **LoRA:** <https://arxiv.org/pdf/2106.09685>
- **LoRA Dropout:** <https://arxiv.org/pdf/2404.09610>
- **PEFT:** <https://arxiv.org/abs/2312.12148>
- **Ultimate Guide to Fine-Tuning LLMs:** <https://arxiv.org/pdf/2408.13296>
- **Merging model:** https://huggingface.co/docs/peft/en/developer_guides/lora#weight-decomposed-low-rank-adaptation-dora
- **Transformer:** <https://arxiv.org/pdf/1706.03762>
- **Fastformer:** <https://arxiv.org/pdf/2108.09084>
- **LlaMA models:** www.llama.com
- **PyTorch container:** <https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-24-09.html>
- **Olivia supercomputer:** <https://www.sigma2.no/meet-olivia-norways-next-supercomputer>
- **Overview of High-Performance Computing (HPC):**
https://github.com/HichamAgueny/HPC_architecture/blob/main/webinar_HPC.pdf

Defining hyperparameter

- A **hyperparameter** is a configurable variable set before training an AI model.
- It controls the learning process such as the **learning rate**, **batch size**, number of **epochs**.
- Its tuning is crucial for enhancing the model’s performance and accuracy.

Hyperparameter	Description	Impact on Training
Learning Rate	Controls how much model weights are updated during training. Used in optimization algorithms like SGD.	- Smaller values: More stable but slower training. - Larger values: Faster updates but risk of overshooting minima or instability.
Batch Size	Number of training samples processed before updating model weights.	- Smaller batches: More updates, may generalize better but noisier. - Larger batches: More stable updates, but require more memory.
Epochs	One complete pass through the full training dataset.	- More epochs: Better learning up to a point; excessive epochs may cause overfitting. - Fewer epochs: May lead to underfitting.

END PART III